

INDEX

Exp No.	Experiment Name	Date	Page No
01	Write a C program for Banker's algorithm for deadlock avoidance.	08/09/2025	2-4
02	Write a C program to simulate the CPU scheduling algorithm- Shortest Job First (SJF).	17/05/2025	5-6
03	Write a C program to simulate the CPU scheduling algorithm- Round Robin.	13/10/2025	7-9
04	Write a C program to simulate the CPU scheduling algorithm - First Come First Serve.	21/10/2025	10-11
05	Write a C program to implement page replacement technique: a) First In First Out replacement. b) Least Recently Used (LRU) replacement. c) Optimal replacement.	11/11/2025	12-17
06	Write a C program to simulate the CPU scheduling algorithm - Priority Queue.	22/12/2025	18-19

Problem No: 1

Problem Name: Write a C program for Banker's algorithm for deadlock avoidance.

Algorithm:

1. Initialize:
 - Calculate Need = Max - Allocation
 - Mark all processes as unfinished.
2. Find a process whose:
 - Need $\leq$ Available
3. If found:
 - Add its allocation to Available
 - Mark it as finished
 - Add it to safe sequence
4. Repeat until all processes finish or no process can proceed.

Source Code:

```
#include <stdio.h>
```

```
int main() {
    // Maximum of 5 processes and 3 resource types; increase if needed
    int p, c, count = 0, i, j;
    int alc[5][3];    // Allocation matrix
    int max[5][3];    // Maximum demand matrix
    int need[5][3];   // Remaining need matrix
    int safe[5];       // Stores safe sequence
    int available[3];  // Available resources
    int done[5];       // Flags for completed processes
    int terminate = 0; // Counter to detect unsafe state

    printf("Enter the number of processes and resources: ");
    scanf("%d %d", &p, &c); // Input number of processes and resource types

    // Input allocation matrix
    printf("Enter allocation of resources (matrix %dx%d):\n", p, c);
    for (i = 0; i < p; i++) {
        for (j = 0; j < c; j++) {
            scanf("%d", &alc[i][j]);
        }
    }

    // Input maximum demand matrix
    printf("Enter maximum resources required (matrix %dx%d):\n", p, c);
```

```

for (i = 0; i < p; i++) {
    for (j = 0; j < c; j++) {
        scanf("%d", &max[i][j]);
    }
}
// Input available resources
printf("Enter available resources:\n");
for (i = 0; i < c; i++) {
    scanf("%d", &available[i]);
}

// Calculate and display need matrix (Need = Max - Allocation)
printf("\nNeed matrix (Remaining resource needs):\n");
for (i = 0; i < p; i++) {
    for (j = 0; j < c; j++) {
        need[i][j] = max[i][j] - alc[i][j];
        printf("%d\t", need[i][j]);
    }
    printf("\n");
}

// Initialize done array to track completed processes
for (i = 0; i < p; i++) {
    done[i] = 0;
}
// Main loop to find safe sequence
while (count < p) {
    for (i = 0; i < p; i++) {
        if (done[i] == 0) {
            // Check if this process can be executed
            for (j = 0; j < c; j++) {
                if (need[i][j] > available[j])
                    break;
            }
            // If all needs can be satisfied
            if (j == c) {
                safe[count] = i;    // Add process to safe sequence
                done[i] = 1;        // Mark process as done
                for (j = 0; j < c; j++) {
                    available[j] += alc[i][j]; // Release allocated resources
                }
                count++;
                terminate = 0;    // Reset terminate counter
            } else {
                terminate++;        // Count how many could not proceed
            }
        }
    }
}

```

```

// If all remaining processes can't proceed => unsafe state
if (terminate == (p - 1)) {
    printf("Safe sequence does not exist. System is in an UNSAFE state.\n");
    break;
}
}
// If system is in safe state, print results
if (terminate != (p - 1)) {
    printf("\nAvailable resources after completion:\n");
    for (i = 0; i < c; i++) {
        printf("%d\t", available[i]);
    }
    printf("\n\nSafe sequence:\n");
    for (i = 0; i < p; i++) {
        printf("P%d\t", safe[i]);
    }
    printf("\n");
}
return 0;
}

```

Sample Input & Output:

```

Enter the number of processes and resources: 5 3
Enter allocation of resources (matrix 5x3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter maximum resources required (matrix 5x3):
7 5 3
3 2 2
9 0 2
4 2 2
5 3 3
Enter available resources:
3 3 2

Need matrix (Remaining resource needs):
7      4      3
1      2      2
6      0      0
2      1      1
5      3      1

Available resources after completion:
10     5     7

Safe sequence:
P1      P3      P4      P0      P2

```

Problem No: 2

Problem Name: Write a C program to simulate the CPU scheduling algorithm-Shortest Job First (SJF).

Algorithm:

1. Input number of processes.
2. Input burst time for each process.
3. Sort processes in ascending order of burst time.
4. Calculate:
 - Waiting Time (WT)
 - Turnaround Time (TAT)
5. Calculate average WT and TAT.
6. Display results.

Source Code:

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], wt[n], tat[n], p[n];

    // Input burst time
    printf("Enter burst time for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d: ", i);
        scanf("%d", &bt[i]);
        p[i] = i;
    }

    // Sort processes by burst time (ascending)
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(bt[i] > bt[j]) {
                // Swap burst time
                int temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                // Swap process IDs
                temp = p[i];
                p[i] = p[j];
            }
        }
    }
}
```

```

        p[j] = temp;
    }
}

// Calculate waiting time
wt[0] = 0;
for(i = 1; i < n; i++) {
    wt[i] = wt[i-1] + bt[i-1];
}

// Calculate turnaround time
for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

// Display results
float avg_wt = 0, avg_tat = 0;

printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t\t%d\t\t%d\n", p[i], bt[i], wt[i], tat[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

Sample Input & Output:

```

Enter number of processes: 4
Enter burst time for each process:
P0: 6
P1: 8
P2: 7
P3: 3

Process Burst Time      Waiting Time      Turnaround Time
P3      3              0                3
P0      6              3                9
P2      7              9               16
P1      8             16               24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00

```

Problem No: 3

Problem Name: Write a C program to simulate the CPU scheduling algorithm- Round Robin.

Algorithm:

1. Input:
 - Number of processes n.
 - Burst time for each process.
 - Time Quantum TQ
2. Initialize:
 - Remaining time = Burst time
 - Waiting time = 0
3. Repeat until all processes finish.
4. Calculate: Turnaround Time = Waiting Time + Burst Time
5. Compute averages and display results.

Source Code:

```
#include <stdio.h>

int main()
{
    int cnt, j, n, t, remain, flag = 0, tq;
    int wt = 0, tat = 0; // Total waiting time and total turnaround time
    int at[10], bt[10], rt[10]; // Arrival Time, Burst Time, Remaining Time arrays

    // Input total number of processes
    printf("Enter Total Process:\t ");
    scanf("%d", &n);
    remain = n; // Number of processes yet to be completed

    // Input arrival and burst time for each process
    for (cnt = 0; cnt < n; cnt++)
    {
        printf("Enter Arrival Time and Burst Time for Process Number %d :", cnt + 1);
        scanf("%d", &at[cnt]); // Arrival time
        scanf("%d", &bt[cnt]); // Burst time
        rt[cnt] = bt[cnt]; // Initialize remaining time
    }

    // Input the time quantum
    printf("Enter Time Quantum:\t");
    scanf("%d", &tq);

    printf("\n\nProcess\t|Turnaround Time|Waiting Time\n\n");
```

```

// Round Robin Scheduling Logic
for (t = 0, cnt = 0; remain != 0;)
{
    // If the process can finish in the given time quantum
    if (rt[cnt] <= tq && rt[cnt] > 0)
    {
        t += rt[cnt]; // Increase current time
        rt[cnt] = 0; // Process completed
        flag = 1; // Flag to indicate process completion
    }
    // If the process needs more time than the time quantum
    else if (rt[cnt] > 0)
    {
        rt[cnt] -= tq; // Decrease remaining time

        t += tq; // Increase current time
    }

    // If process is completed, print its turnaround and waiting time
    if (rt[cnt] == 0 && flag == 1)
    {
        remain--; // Decrement remaining process count
        printf("P[%d]\t\t%d\t\t%d\n", cnt + 1, t - at[cnt], t - at[cnt] - bt[cnt]);
        wt += t - at[cnt] - bt[cnt]; // Add to total waiting time
        tat += t - at[cnt]; // Add to total turnaround time
        flag = 0; // Reset flag
    }

    // Move to the next process
    // If at the last process, loop back to first
    if (cnt == n - 1)
        cnt = 0;
    // Else, go to the next process if its arrival time has passed
    else if (at[cnt + 1] <= t)
        cnt++;
    // If the next process hasn't arrived, start again from the first process
    else
        cnt = 0;
}

// Print average waiting and turnaround times
printf("\nAverage Waiting Time= %f\n", wt * 1.0 / n);
printf("Avg Turnaround Time = %f", tat * 1.0 / n);
return 0;
}

```

Sample Input & Output:

```

Enter Total Process:      4
Enter Arrival Time and Burst Time for Process Number 1 :0 5
Enter Arrival Time and Burst Time for Process Number 2 :1 4
Enter Arrival Time and Burst Time for Process Number 3 :2 2
Enter Arrival Time and Burst Time for Process Number 4 :4 1
Enter Time Quantum:      2

Process |Turnaround Time|Waiting Time
P[3]    |          4    |          2
P[4]    |          3    |          2
P[2]    |         10    |          6
P[1]    |         12    |          7

Average Waiting Time= 4.250000
Avg Turnaround Time = 7.250000
Process returned 0 (0x0)   execution time : 52.328 s
Press any key to continue.

```

Problem No: 4

Problem Name: Write a C program to simulate the CPU scheduling algorithm - First Come First Serve.

Algorithm:

1. Input number of processes n.
2. Input burst time (BT) for each process.
3. Assume processes are executed in the order they are entered.
4. Calculate:
 - Waiting Time (WT)
 - Turnaround Time (TAT)
5. Calculate average WT and TAT.
6. Display results.

Source Code:

```
#include <stdio.h>
```

```
int main() {
    int n, i;
    int bt[20], wt[20], tat[20]; // bt = Burst Time, wt = Waiting Time, tat = Turnaround Time
    float avg_wt = 0, avg_tat = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    // Input burst times
    printf("Enter Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P[%d]: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // First process has 0 waiting time
    wt[0] = 0;

    // Calculate waiting time for each process
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Calculate turnaround time for each process
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
}
```

```

// Print process details
printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
for (i = 0; i < n; i++) {
    printf("P[%d]\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
}

// Calculate average times
avg_wt /= n;
avg_tat /= n;

printf("\nAverage Waiting Time: %.2f", avg_wt);
printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
return 0;
}

```

Sample Input & Output:

```

Enter the number of processes: 3
Enter Burst Time for each process:
P[1]: 3
P[2]: 4
P[3]: 5

Process Burst Time      Waiting Time      Turnaround Time
P[1]    3              0                3
P[2]    4              3                7
P[3]    5              7               12

Average Waiting Time: 3.33
Average Turnaround Time: 7.33

Process returned 0 (0x0)   execution time : 19.273 s
Press any key to continue.

```

Problem No: 5

Problem Name: Write a C program to implement page replacement technique:

- a) First In First Out replacement.
- b) Least Recently Used (LRU) replacement.
- c) Optimal replacement.

Algorithm:

First In First Out replacement:

1. Start with an empty set of frames.
2. For each page in the reference string:
 - If the page is already in memory → Hit
 - Else → Page Fault
3. Count total page faults.

Least Recently Used (LRU) replacement:

1. Start with empty frames.
2. For each page in the reference string:
 - If the page is already in memory → Hit
 - Else → Page Fault
3. Keep track of usage using a counter or timestamps.
4. Count total page faults.

Optimal replacement:

1. Start with empty frames.
2. For each page in the reference string:
 - If the page is already in memory → Hit
 - Else → Page Fault
3. Count total page faults.

Source Code:

First In First Out replacement:

```
#include <stdio.h>
```

```
int main() {
    int frames, pages;
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int ref[pages];
    printf("Enter reference string: ");
    for(int i = 0; i < pages; i++) {
        scanf("%d", &ref[i]);
    }
    int frame[frames];
    int front = 0; // Points to oldest page
    int faults = 0;
    // Initialize frames with -1 (empty)
    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
    }
}
```

```

}

printf("\nPage\tFrames\n");
for(int i = 0; i < pages; i++) {
    int found = 0;
    // Check if page already exists
    for(int j = 0; j < frames; j++) {
        if(frame[j] == ref[i]) {
            found = 1;
            break;
        }
    }
    // If not found → page fault
    if(!found) {
        frame[front] = ref[i];
        front = (front + 1) % frames;
        faults++;
    }
    // Display frames
    printf("%d\t", ref[i]);
    for(int j = 0; j < frames; j++) {
        if(frame[j] != -1)
            printf("%d ", frame[j]);
        else
            printf("- ");
    }
    printf("\n");
}
printf("\nTotal Page Faults = %d\n", faults);
return 0;
}

```

Least Recently Used (LRU) replacement:

```
#include <stdio.h>
```

```

int main() {
    int frames, pages;
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int ref[pages];
    printf("Enter reference string: ");
    for(int i = 0; i < pages; i++) {
        scanf("%d", &ref[i]);
    }
    int frame[frames], time[frames];
    int faults = 0, counter = 0;
    // Initialize frames
    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = 0;
    }
}

```

```

printf("\nPage\tFrames\n");
for(int i = 0; i < pages; i++) {
    int found = 0;

    // Check for hit
    for(int j = 0; j < frames; j++) {
        if(frame[j] == ref[i]) {
            counter++;
            time[j] = counter; // Update recent use
            found = 1;
            break;
        }
    }

    // Page fault
    if(!found) {
        int pos = 0;

        // Find least recently used page
        for(int j = 1; j < frames; j++) {
            if(time[j] < time[pos]) {
                pos = j;
            }
        }

        frame[pos] = ref[i];
        counter++;
        time[pos] = counter;
        faults++;
    }

    // Display frames
    printf("%d\t", ref[i]);
    for(int j = 0; j < frames; j++) {
        if(frame[j] != -1)
            printf("%d ", frame[j]);
        else
            printf("- ");
    }
    printf("\n");
}
printf("\nTotal Page Faults = %d\n", faults);
return 0;
}

```

Optimal replacement:

```
#include <stdio.h>
```

```

int main() {
    int frames, pages;
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    int ref[pages];

```

```

printf("Enter reference string: ");
for(int i = 0; i < pages; i++) {
    scanf("%d", &ref[i]);
}
int frame[frames];
int faults = 0;
// Initialize frames
for(int i = 0; i < frames; i++) {
    frame[i] = -1;
}

printf("\nPage\tFrames\n");

for(int i = 0; i < pages; i++) {
    int found = 0;
    // Check for hit
    for(int j = 0; j < frames; j++) {
        if(frame[j] == ref[i]) {
            found = 1;
            break;
        }
    }
    // Page fault
    if(!found) {
        int pos = -1, farthest = i + 1;

        for(int j = 0; j < frames; j++) {
            int k;
            for(k = i + 1; k < pages; k++) {
                if(frame[j] == ref[k]) {
                    if(k > farthest) {
                        farthest = k;
                        pos = j;
                    }
                }
            }
            break;
        }
        // If page not found in future
        if(k == pages) {
            pos = j;
            break;
        }
    }
    // If all pages will be used, replace farthest
    if(pos == -1) {
        pos = 0;
    }
    frame[pos] = ref[i];
    faults++;
}
// Display frames
printf("%d\t", ref[i]);
for(int j = 0; j < frames; j++) {
    if(frame[j] != -1)
        printf("%d ", frame[j]);
}

```

```

        else
            printf("- ");
    }
    printf("\n");
}
printf("\nTotal Page Faults = %d\n", faults);
return 0;
}

```

Sample Input & Output:

First In First Out replacement:

```

Enter number of frames: 3
Enter number of pages: 9
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page   Frames
7       7 - -
0       7 0 -
1       7 0 1
2       2 0 1
0       2 0 1
3       2 3 1
0       2 3 0
4       4 3 0
2       4 2 0

Total Page Faults = 8

```

Least Recently Used (LRU) replacement:

```

Enter number of frames: 3
Enter number of pages: 9
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page   Frames
7       7 - -
0       7 0 -
1       7 0 1
2       2 0 1
0       2 0 1
3       2 0 3
0       2 0 3
4       4 0 3
2       4 0 2

Total Page Faults = 7

```

Optimal replacement:

```
Enter number of frames: 3
Enter number of pages: 9
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2
```

Page	Frames
7	7 - -
0	0 - -
1	0 1 -
2	0 2 -
0	0 2 -
3	0 2 3
0	0 2 3
4	4 2 3
2	4 2 3

Total Page Faults = 6

Problem No: 6

Problem Name: Write a C program to simulate the CPU scheduling algorithm - Priority Queue.

Algorithm:

1. Input number of processes n.
2. Input:
 - Burst Time (BT).
 - Priority (P).
3. Assign process IDs.
4. Sort processes based on priority (ascending).
5. Calculate:
 - Waiting Time (WT).
 - Turnaround Time (TAT).
6. Compute averages.
7. Display results.

Source Code:

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], pr[n], wt[n], tat[n], p[n];

    // Input burst time and priority
    printf("Enter Burst Time and Priority for each process:\n");
    for(i = 0; i < n; i++) {
        printf("P%d (BT Priority): ", i);
        scanf("%d %d", &bt[i], &pr[i]);
        p[i] = i;
    }

    // Sort based on priority (lower value = higher priority)
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(pr[i] > pr[j]) {
                // Swap priority
                int temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;

                // Swap burst time
                temp = bt[i];
```

```

        bt[i] = bt[j];
        bt[j] = temp;

        // Swap process ID
        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}

// Calculate waiting time
wt[0] = 0;
for(i = 1; i < n; i++) {
    wt[i] = wt[i-1] + bt[i-1];
}

// Calculate turnaround time
for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

// Display results
float avg_wt = 0, avg_tat = 0;
printf("\nProcess\tPriority\tBurst Time\tWaiting Time\tTurnaround Time\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i], pr[i], bt[i], wt[i], tat[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
}
printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);
return 0;
}

```

Sample Input & Output:

```

Enter number of processes: 4
Enter Burst Time and Priority for each process:
P0 (BT Priority): 5 2
P1 (BT Priority): 3 1
P2 (BT Priority): 8 4
P3 (BT Priority): 6 3

Process Priority      Burst Time    Waiting Time   Turnaround Time
P1      1             3             0              3
P0      2             5             3              8
P3      3             6             8             14
P2      4             8            14             22

Average Waiting Time = 6.25
Average Turnaround Time = 11.75

Process returned 0 (0x0)   execution time : 43.854 s
Press any key to continue.

```